

Bufory

Tablice

- Tablica (ang. array) to spójny obszar pamięci podzielony na komórki.
- Komórki w tablicy są numerowane liczbami całkowitymi począwszy od 0.
- Zaleta: czas dostępu do każdej komórki w tablicy jest stały $O(1)$.
- Wada: tablica jest strukturą nierozszerzalną.
- Zastosowanie: tam, gdzie można z góry określić ilość danych.
- Oznaczenie: `TYP tablica[n]`

Tablice dynamiczne

- Tablica dynamiczna (ang. resizable array) dopasowuje swój rozmiar do rozmiaru danych tak, aby proporcja wykorzystanych komórek pamięci w stosunku do pojemności tablicy była ograniczona przez pewne stałe.
- Operacje na tablicy dynamicznej:
 - Dostęp do slotów w obrębie zapełnionego fragmentu tablicy.
 - Dodanie nowej komórki na końcu tablicy (może się to wiązać z utworzeniem nowej dwukrotnie większej tablicy i z przepisaniem danych).
 - Usunięcie komórki z końca tablicy (może się to wiązać z utworzeniem nowej dwukrotnie mniejszej tablicy i z przepisaniem danych).
- Modyfikacje: rozważa się też tablice dynamiczne, do których można wstawiać i usuwać elementy z przodu.

Tablice dynamiczne

- Idea struktury:
 - Trzymamy tablicę wraz z dodatkowymi parametrami:
 - pojemność (liczba wszystkich slotów w tablicy),
 - wypełnienie (liczba wykorzystanych slotów w tablicy).
 - W przypadku, gdy rozważamy tablicę dynamiczną, do której można dopisywać i usuwać elementy zarówno na końcu jak i z przodu wymagany jest jeszcze jeden dodatkowy parametr:
 - początek (pozycja, od której zaczynają się wypełnione sloty w tablicy; sama tablica będzie wtedy zawinięta).
 - Gdy tablica się zapełni w 100% i chcemy wstawić następny element, to powiększamy ją dwukrotnie.
 - Gdy wypełnienie tablicy spadnie do 25% i chcemy usunąć kolejny element, to pomniejszamy ją dwukrotnie.

Tablice dynamiczne

- Czasy operacji na tablicy dynamicznej:
 - Dostęp do zapełnionych slotów $O(1)$
 - Dodanie nowej komórki na końcu tablicy $O(n)$.
 - Usunięcie komórki z końca tablicy $O(n)$.
 - Uwaga: przeważnie dodanie/usunięcie komórki wymaga czasu $O(1)$.
- Analiza zamortyzowana:
 - Za pomocą metody księgowania jednostek kredytowych – każda operacja dodawania/usuwania elementu do/z tablicy pozostawia 2 kredyty czasowe do wykorzystania przy przepisywaniu tablicy.
 - Zamortyzowany koszt każdej pojedynczej operacji na tablicy dynamicznej jest stały $O(1)$ i wynosi 3 jednostki czasowe.

Tablice dynamiczne

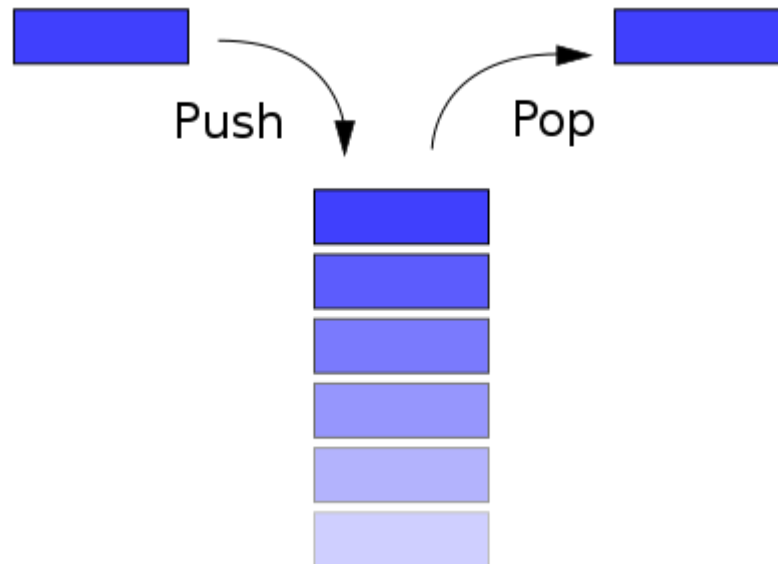
- Modyfikacje: tablica dynamiczna może się rozszerzać w obu kierunkach.
- Zastosowanie: nie można z góry określić rozmiaru danych, ale potrzebujemy szybkiego dostępu do każdej komórki w tablicy.
- Realizacja tej struktury w bibliotece standardowej C++: `vector<>`, `deque<>`.

Stos

- Stos (ang. stack) to struktura, która pozwala na gromadzenie i przetwarzanie danych zgodnie z kolejnością ich wejścia do struktury: element który został najpóźniej dodany do struktury zostanie z niej najszybciej usunięty (ang. LIFO – Last In, First Out; am. LCFS – Last Come, First Served).
- Analogia: stos książek na biurku, ubrania na krześle .

Stos

- Operacje na stosie:
 - Włożenie elementu na szczyt stosu (ang. **push**)
 - Usunięcie elementu ze szczytu stosu (ang. **pop**)
 - Podglądnięcie elementu na szczycie stosu (ang. **top**)
 - Liczba wszystkich elementów na stosie (ang. **size**)



Implementacja stosu na tablicy

- Do zwykłej tablicy dokładamy dwie informacje:
pojemność stosu (liczba slotów w tablicy), zapełnienie
stosu (liczba użytych elementów w tablicy):
 - `tab[]` – tablica na dane
 - `poj` – wielkość tablicy
 - `zap` – zapełnienie tablicy

Implementacja stosu na tablicy

- Operacja push(x):

```
push(x) {  
    if (zap = poj) error;  
    tab[zap] := x;  
    inc(zap); // zap += 1  
}
```

- Operacja pop():

```
pop() {  
    if (zap = 0) error;  
    dec(zap); // zap -= 1  
    x := tab[zap]; tab[zap] := null;  
    return x;  
}
```

Implementacja stosu na tablicy

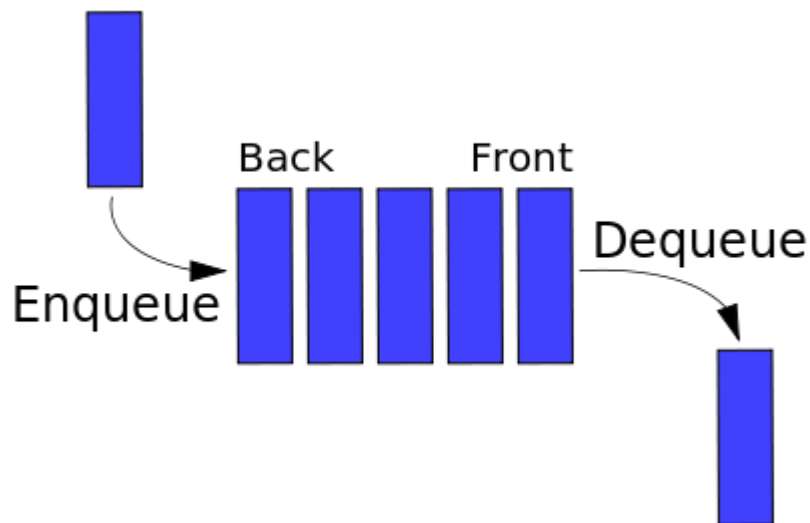
- Wszystkie operacje na stosie zaimplementowanym na tablicy zajmują czas stały $O(1)$.
- Implementacja stosu na liście jest tak samo efektywna.

Kolejka

- Kolejka (ang. queue) to struktura, która pozwala na gromadzenie i przetwarzanie danych zgodnie z kolejnością ich wejścia do struktury: element który został najwcześniej dodany do struktury zostanie z niej najszybciej usunięty (ang. FIFO – First In, First Out; am. FCFS – First Come, First Served).
- Analogia: kolejka do kasy, taśma w fabryce.

Kolejka

- Operacje na kolejce:
 - Włożenie elementu na koniec kolejki (ang. **enqueue**)
 - Usunięcie elementu z początku kolejki (ang. **dequeue**)
 - Podglądnięcie elementu na początku kolejki (ang. **front**)
 - Podglądnięcie elementu na końcu kolejki (ang. **back**)
 - Liczba wszystkich elementów w kolejce (ang. **size**)



Implementacja kolejki na tablicy

- Do zwykłej tablicy dokładamy trzy informacje:
pojemność kolejki (liczba slotów w tablicy), początek kolejki (numer indeksu komórki, w którym znajduje się pierwszy element w kolejce), wypełnienie kolejki (liczba użytych elementów w tablicy):
 - `tab[]` – tablica na dane
 - `poj` – wielkość tablicy
 - `zap` – wypełnienie tablicy
 - `pocz` – miejsce, od którego rozpoczyna się kolejka
- Uwaga: zawijamy tablicę.

Implementacja kolejki na tablicy

- Operacja enqueue(x):

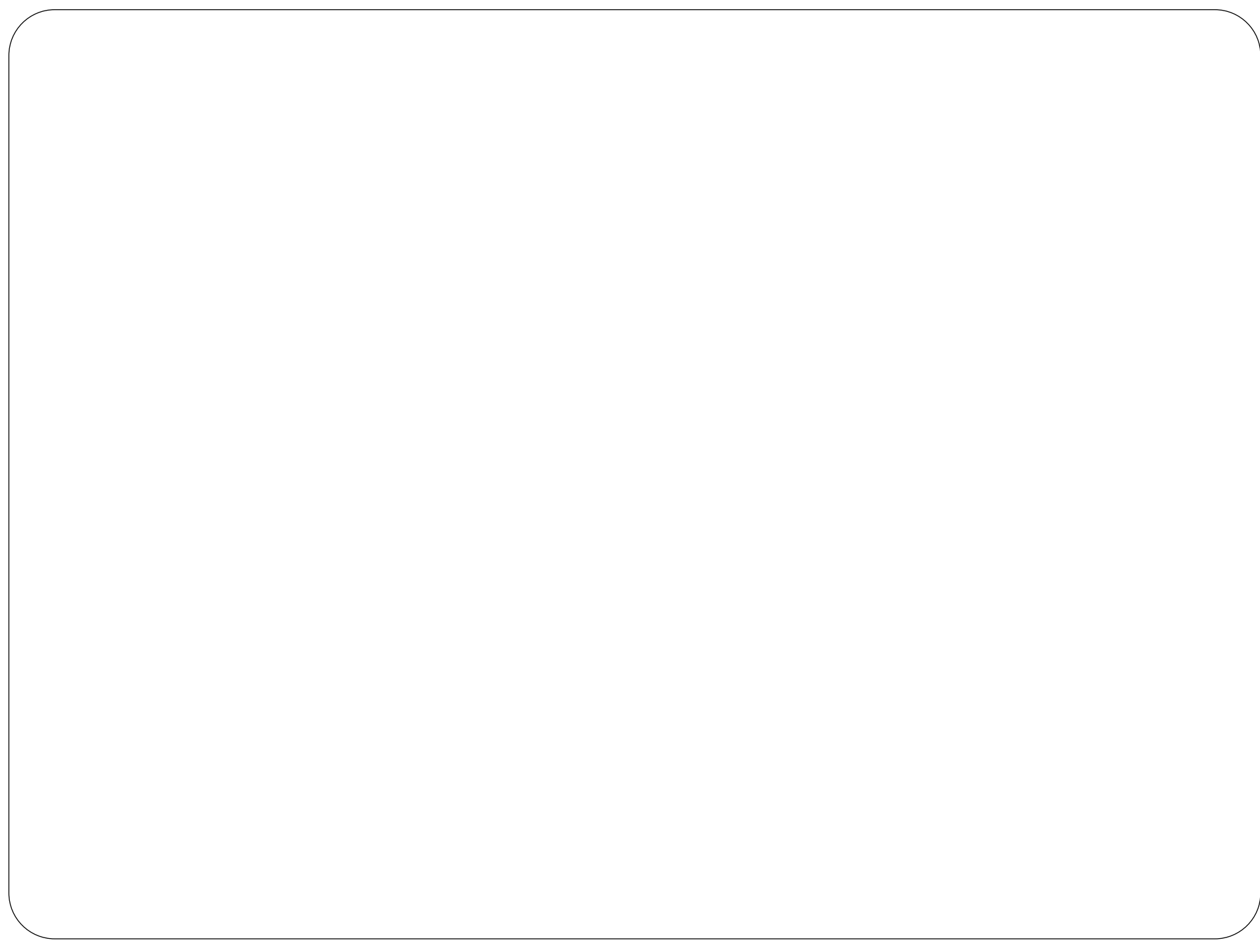
```
enqueue(x) {  
    if (zap = poj) error;  
    tab[(pocz + zap) mod poj] := x;  
    inc(zap); // zap += 1  
}
```

- Operacja dequeue():

```
dequeue() {  
    if (zap = 0) error;  
    x := tab[pocz]; tab[pocz] := null;  
    pocz = (pocz + 1) mod poj;  
    dec(zap); // zap -= 1  
    return x;  
}
```

Implementacja kolejki na tablicy

- Wszystkie operacje na kolejce zaimplementowanej na tablicy zajmują czas stały $O(1)$.
- Implementacja kolejki na liście jest tak samo efektywna.



Zadania i problemy

- Jak zasymulować działanie kolejki mając do dyspozycji dwa stosy?
- Jak zmodyfikować działanie tablicy dynamicznej, aby każda operacja powiększania i pomniejszania tablicy o jeden element działała w pesymistycznym czasie stałym $O(1)$?